Building_Energy_Modeling_Parallel_IDF_Preparation_Technical_Reference

Parallel IDF Preparation — Technical Reference

Applies to: Option 7i (single-building) and Option 9i (neighbourhood) batch runs

---

Overview

Batch simulation runs in two distinct parallel phases:

1. **PREP phase** — IDF files are generated/modified in parallel (ProcessPoolExecutor)
2. **SIM phase** — Generated IDFs are submitted to EnergyPlus in parallel (ThreadPoolExecutor on Windows)

The two phases are separated by design: all IDFs must be written to disk before any E+ process starts. This avoids I/O contention and makes failure recovery straightforward.

Phase 1 — Parallel IDF Preparation

Entry point

```
MAX_PREP_WORKERS = int(os.environ.get("BEM_MAX_PREP_WORKERS", config.MAX_NEIGHBOURHOOD_WORKERS))
```

The configured worker cap defaults to `8`. Override at runtime:

```
BEM_MAX_PREP_WORKERS=4 python main_BEM.py
```

Pool function

```
def _run_prep_pool(jobs: list, max_workers: int, worker_fn=None) -> list:
```

- Accepts a flat list of job dicts.
- Falls back to **serial** when `max_workers == 1` or `len(jobs) == 0`.
- Uses `concurrent.futures.ProcessPoolExecutor` for true multiprocessing (required because eppy mutates class-level globals — threads would race).
- Submits all jobs up-front: `{executor.submit(worker_fn, job): job for job in jobs}`.
- Collects results with `as_completed(futures)` — completion order is non-deterministic, which is fine because each job is fully self-contained.
- **Worker crashes** (BrokenProcessPool, OOM, etc.) trigger an immediate `RuntimeError` re-raise that aborts the entire prep phase.
- **Application-level errors** (bad IDF path, applier exception) are caught inside the worker and returned as `ok=False` in the result dict — the pool itself does not crash.

Worker functions

There are two module-level worker functions (module-level is mandatory for Windows `spawn`-mode pickle):

`_prep_one_scenario` — Option 7 (single buildings)

Input: a dict with keys `idf_path`, `scenario_key`, `mod_idf_path`, `idd_path`, `ep_version`, `pv_mode`, etc.

Logic per worker:

1. Depending on `scenario_key`: copy baseline (`DEFAULT`), apply IAL, apply HPENV construction swap, apply EEM1/2/3 applier chain.
2. Run `idf_optimizer.optimize_idf()` (removes redundant objects, sorts sections).
3. Apply Tier-3 PV injector if `pv_mode == 'improve'`.
4. Capture all stdout via `contextlib.redirect_stdout(io.StringIO())` — the main process prints it atomically after the future resolves.

Returns: `{"ok": True/False, "worker_log": str, "mod_idf_path": str, ...}`.

`_prep_one_nu_scenario` — Option 9 (neighbourhood)

Thin adapter around `_prepare_neighbourhood_case_artifacts()`. Same stdout-capture and ok/error contract as above.

Logging pattern

Workers never write directly to stdout (they are separate processes — output would interleave). Instead:

```
buf = io.StringIO()
with contextlib.redirect_stdout(buf):
    # ... all work happens here
return {"worker_log": buf.getvalue(), ...}
```

Main process prints atomically after each future:

```
if result.get("worker_log"):
    sys.stdout.write(result["worker_log"])
print(f"  [PREP {done}/{total}] {result['batch_name']} {result['scenario_key']}")
```

This produces clean, ordered `[PREP N/M]` lines even though completions are non-deterministic.

Job dict structure (Option 7 example)

```
{
    "idf_path":           "<path to source IDF>",
    "scenario_key":       "EEM_J_ENVELOPE",      # or DEFAULT / IAL / HPENV / EEM_J_ENV_HVAC / EEM_J_ENV_HVAC_DHW
    "scenario_dir_label": "EEM1",
    "mod_idf_path":       "<path to output IDF>",
    "modified_dir":       "<parent output dir>",
    "source_name":        "MediumOffice",
    "idd_path":           "<path to .idd>",
    "ep_version":         "23.1",
    "pv_mode":            "improve",              # or None
    "batch_name":         "MediumOffice_CAN_MTL",
    "scenario_dir":       "<full scenario dir path>",
    "base_short_name":    "MediumOffice",
    "parent_dir":         "<parent>",
}
```

Phase 2 — Parallel E+ Simulation

Entry point

```
def run_simulations_parallel(simulation_jobs, ep_path, max_workers=None):
```

`max_workers` defaults to `os.cpu_count()` and is capped at `len(simulation_jobs)`.

Executor choice

```
ExecutorClass = ThreadPoolExecutor if platform.system() == "Windows" else ProcessPoolExecutor
```

EnergyPlus runs as a **subprocess** (I/O-bound, not CPU-bound from Python's perspective). On Windows, `ThreadPoolExecutor` is used because:

- Subprocess spawning from a child `ProcessPoolExecutor` process on Windows is expensive and sometimes deadlocks.
- Threads release the GIL during `subprocess.run()` — parallelism is real.

On Linux/macOS the pool is `ProcessPoolExecutor` for isolation.

Each worker job carries `n_jobs=1` so E+ itself uses one CPU thread — this avoids over-subscription when many simulations run concurrently.

Job dict structure

```
{
    "idf":        "<path to prepped IDF>",
    "epw":        "<path to weather file>",
    "output_dir": "<per-job output directory>",
    "name":       "MediumOffice_EEM1",
    "ep_path":    "<path to EnergyPlus executable>",
    "n_jobs":     1,
    "quiet":      True,
}
```

Progress output

```
[42/120] [OK]   MediumOffice_EEM1 (04:31)
[43/120] [FAIL] RetailStripmall_EEM3 (04:32)
```

Format: `[completed/total] [status] job_name (elapsed_mm:ss)`.

Memory budget

Component	RAM per worker
IDF prep worker (eppy + applier state)	~250-450 MB
E+ sim worker (subprocess, not Python)	~100-300 MB per E+ process

Rule of thumb for prep workers:

System RAM	Recommended MAX_PREP_WORKERS
16 GB	2-3
32 GB	4-5
64 GB	8

Design constraints (why it's built this way)

Constraint	Reason
`ProcessPoolExecutor` for IDF prep	`eppy` stores parsed IDD as class-level globals; threads from the same process share these and race on writes
Module-level worker functions	Windows uses `spawn` (not `fork`) for new processes — only picklable objects cross the process boundary; closures and lambdas cannot be pickled
Stdout capture inside workers	Child processes have separate stdout; direct `print()` would either be lost or interleave with main process output
PREP before SIM, not interleaved	Avoids disk contention; a failed PREP aborts cleanly before any E+ license slot is consumed
`ok=False` vs exception	Application errors (bad IDF, applier failure) return a result dict so the remaining jobs complete; only executor crashes (OOM, segfault) are re-raised

---

Adapting this pattern to another project

Minimum viable parallel prep for a different EnergyPlus pipeline:

```python
# 1. imports
from concurrent.futures import ProcessPoolExecutor, as_completed
import contextlib, io, sys

# 2. worker — must be module-level
def _prep_worker(args: dict) -> dict:
    buf = io.StringIO()
    with contextlib.redirect_stdout(buf):
        try:
            # ... your IDF modification logic here ...
            ok, error = True, None
        except Exception as e:
            ok, error = False, str(e)
    return {"label": args["label"], "out_path": args["out_path"],
            "ok": ok, "error": error, "worker_log": buf.getvalue()}

# 3. pool runner
def run_prep_pool(jobs: list, max_workers: int) -> list:
    if max_workers == 1 or not jobs:
        return [_prep_worker(j) for j in jobs]
    results, done = [], 0
    with ProcessPoolExecutor(max_workers=max_workers) as ex:
        futures = {ex.submit(_prep_worker, j): j for j in jobs}
        for f in as_completed(futures):
            exc = f.exception()
            if exc:
                raise RuntimeError(f"Prep worker crashed: {exc}") from exc
            r = f.result()
            done += 1
            if r.get("worker_log"):
                sys.stdout.write(r["worker_log"])
            print(f"  [PREP {done}/{len(jobs)}] {r['label']} ok={r['ok']}")
            results.append(r)
    return results

# 4. build job list and call
jobs = [{"label": f"building_{i}", "out_path": f"out_{i}.idf", ...} for i in range(N)]
results = run_prep_pool(jobs, max_workers=8)

# 5. handle failures before simulating
failed = [r for r in results if not r["ok"]]
if failed:
    for r in failed: print(f"  PREP FAILED: {r['label']} -- {r['error']}")
    raise SystemExit("Prep phase had failures — aborting before simulation.")

sim_jobs = [{"idf": r["out_path"], ...} for r in results if r["ok"]]
```
